GPU-Accelerated Portfolio Optimization for Multi-Asset Allocation using CUDA

Stefan Mauch
EN.605.617 – Introduction to GPU Programming
Johns Hopkins University | Spring 2026

ABSTRACT

This paper presents a GPU-accelerated Global Minimum Variance (GMV) portfolio optimizer implemented in C++17/CUDA, benchmarked against a serial CPU baseline using hand-coded Cholesky factorization. The GPU pipeline integrates cuBLAS (BLAS-3 Dgemm for covariance estimation) and cuSOLVER (Dpotrf/Dpotrs for Cholesky-based linear system solving) on an NVIDIA Tesla T4 (Compute Capability 7.5, 40 SMs, 15.64 GB GDDR6, CUDA 12.8). Scalability experiments across portfolio sizes $N \in \{25, 50, 100, 250, 500, 1000\}$ with $T = 2{,}000$ synthetic observations reveal a fixed GPU initialization overhead of ≈220–305 ms attributable to cuBLAS/cuSOLVER handle creation and host-to-device memory transfer. The GPU break-even point occurs at $N \approx 248$ assets (interpolated from N=100 and N=250 data points). At $N = 500$, the GPU achieves 3.875× speedup (CPU: 880.555 ms; GPU: 227.218 ms); at $N = 1{,}000$ it reaches 15.258× (CPU: 4,960.695 ms; GPU: 325.117 ms). Numerical accuracy is exact across all sizes: $\max|w_CPU - w_GPU| = 0.00e{+}00$ and $\Delta\sigma^2/\sigma^2 = 0.00e{+}00$ in all cases. The framework is validated on 44 ETFs spanning eight asset classes (2,011 trading days, Jan 2018–Dec 2025). Portfolio analytics include Monte Carlo efficient frontier (50,000 portfolios), rolling out-of-sample backtest (GMV: Ann. Ret. = 9.3%, Ann. Vol. = 11.6%, Sharpe = 0.80, MaxDD = −24.9%), and pairwise correlation/covariance structure analysis.

Keywords: GPU Computing, CUDA, cuBLAS, cuSOLVER, Portfolio Optimization, Global Minimum Variance, Cholesky Factorization, cuSOLVER Dpotrf, High-Performance Computing

1. Introduction

Portfolio construction is a fundamental problem in quantitative finance. Markowitz's mean–variance framework [1] prescribes allocating capital across assets to minimize portfolio risk. In the special case of the Global Minimum Variance (GMV) portfolio, variance is minimized unconditionally subject only to the full-investment constraint. The resulting closed-form solution depends entirely on the asset covariance matrix, which must be estimated from historical return data and then used to solve a linear system—a pipeline that becomes computationally limiting for institutional-scale portfolios operating at high frequency or with large asset universes.

Graphics Processing Units offer massively parallel architectures ideally suited to dense linear algebra. NVIDIA's CUDA programming model, combined with the cuBLAS and cuSOLVER libraries, provides GPU-accelerated BLAS-3 matrix multiplication and direct Cholesky solvers that underpin the GMV pipeline. The central research question is: at what portfolio size does GPU acceleration become beneficial, and what speedup is achievable at institutional scale?

This project makes four concrete contributions: (1) a complete C++17/CUDA implementation of the GMV pipeline with custom CUDA kernels and cuBLAS/cuSOLVER integration; (2) rigorous scalability benchmarking across six portfolio sizes from $N = 25$ to $N = 1{,}000$; (3) identification of the GPU break-even at $N \approx 248$ and 15.258× speedup at $N = 1{,}000$; and (4) financial validation on 44 ETFs with Monte Carlo efficient frontier and rolling backtest analysis.

2. Background and Related Work

2.1 Mean–Variance Portfolio Theory

Markowitz [1] established the efficient frontier as the set of portfolios minimizing variance for a given expected return. The full mean–variance formulation with risk aversion parameter $\lambda \geq 0$ is:

$$\min_{w} \ \frac{1}{2} w^{\mathrm{T}} \Sigma w - \lambda \mu^{\mathrm{T}} w \quad \text{subject to} \quad \mathbf{1}^{\mathrm{T}} w = 1 \tag{1}$$

where $w \in \mathbb{R}^R$ is the portfolio weight vector, $\Sigma \in \mathbb{R}^{R \times R}$ is the return covariance matrix, and $\mu \in \mathbb{R}^R$ is the expected return vector. Setting $\lambda = 0$ recovers the GMV problem:

$$\min_{w} \ w^{\mathrm{T}} \Sigma w \quad \text{subject to} \quad \mathbf{1}^{\mathrm{T}} w = 1 \tag{2}$$

2.2 Closed-Form GMV Solution via KKT Conditions

Forming the Lagrangian and applying stationarity and primal feasibility conditions:

$$L(w,\lambda) = w^{\mathrm{T}}\Sigma w + \lambda(\mathbf{1}^{\mathrm{T}}w - 1) \tag{3}$$

The KKT stationarity condition ∇_w L = 0 gives:

$$2\Sigma w + \lambda\mathbf{1} = 0 \implies w = -\frac{\lambda}{2}\Sigma^{-1}\mathbf{1} \tag{4}$$

Substituting the budget constraint ′′TJw = 1 yields the closed-form GMV solution:

$$w^{*} = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^{\mathrm{T}}\Sigma^{-1}\mathbf{1}} \tag{5}$$

This closed-form eliminates the need for an iterative solver: only a single linear system solve $\Sigma x = \varrho$ followed by scalar normalization $w^* = x / (\varrho^T x)$ is required, making it well-suited for GPU acceleration via cuBLAS and cuSOLVER.

2.3 GPU Linear Algebra

cuBLAS [3] provides GPU-optimized BLAS operations including cublasDgemm (BLAS-3 matrix–matrix multiply), which achieves near-peak throughput on Tensor Cores through shared-memory tiling. cuSOLVER [4] provides cusolverDnDpotrf (Cholesky factorization) and cusolverDnDpotrs (triangular solve)—GPU equivalents of LAPACK's dpotrf/dpotrs. The Tesla T4 GPU [2] has 40 SMs, 2,560 CUDA cores, 15.64 GB GDDR6 memory at 256-bit width (≈320 GB/s bandwidth), and Compute Capability 7.5.

3. Financial Model

3.1 Return Computation

Daily simple returns are computed from adjusted closing prices as:

$$r_{t,i} = \frac{P_{t,i} - P_{t-1,i}}{P_{t-1,i}} \tag{6}$$

where P_{t,i} is the adjusted closing price of asset i on day t. The return matrix $R \in \mathbb{R}^{T\times R}$ is stored in column-major (Fortran) order throughout the implementation to maintain BLAS/LAPACK compatibility.

3.2 Covariance Estimation

The sample covariance matrix is computed using Bessel's correction:

$$\Sigma = \frac{1}{T-1} \tilde{R}^T \tilde{R}, \quad \tilde{R} = R - \mathbf{1}\hat{\mu}^T \tag{7}$$

where $\tilde{R} = R - \mathbf{1}\hat{\mu}^T$ is the mean-centered return matrix and $\hat{\mu}$ is the sample mean vector. The matrix form $\Sigma = (1/(T-1))\, \tilde{R}^T\tilde{R}$ maps directly to a cuBLAS BLAS-3 matrix–matrix multiply (cublasDgemm), enabling efficient GPU computation.

3.3 Tikhonov Regularization

Sample covariance matrices estimated from financial return data can be nearly singular, particularly when T is close to N (the Marčenko–Pastur regime). Cholesky factorization requires strict positive definiteness. Tikhonov regularization is applied:

$$\Sigma_{\text{reg}} = \Sigma + \varepsilon I, \quad \varepsilon = 10^{-6} \tag{8}$$

The parameter $\varepsilon = 10^{-6}$ is chosen to be negligible relative to typical daily return variances (order 10^{-4} for equity ETFs) while guaranteeing numerical positive definiteness. This is implemented via a custom CUDA kernel (AddDiagonalKernel) that adds ε to each diagonal element in parallel, with one GPU thread per diagonal entry.

3.4 Portfolio Performance Metrics

The realized portfolio variance at the optimal weights is:

$$\sigma_p^2 = w^{*T} \Sigma\, w^* \tag{9}$$

Annualized risk-adjusted performance is characterized by the Sharpe ratio (risk-free rate = 0):

$$\text{SR} = \frac{252 \cdot \text{E}(r_p)}{\sqrt{252} \cdot \sigma(r_p)} \tag{10}$$

Maximum drawdown is defined as MDD = min_t (V_t − max_{s≤t}V_s) / max_{s≤t}V_s, where V_t is the portfolio cumulative value at time t.

4. System Architecture

4.1 Source Files and Responsibilities

The project is a modular C++17/CUDA application with clear separation of concerns across seven source files compiled with g++ (for .cpp) and nvcc (for .cu):

File	Compiler	Responsibility
data_loader.cpp	g++	CSV parsing, return computation r_{t,i}, column-major packing
utils.cpp	g++	Idx(row,col,ld), SumVector(), DotProduct(), PrintWeights()
cpu_portfolio.cpp	g++	CenterColumns, Covariance, Cholesky, ForwardSub, BackSub
benchmark.cpp	g++	RunBenchmark(): CPU+GPU timing, speedup, max|Δw| computation
gpu_portfolio.cu	nvcc	cuBLAS Dgemv/Dgemm, CenterColumnsKernel, AddDiagonalKernel, cuSOLVER
main.cu	nvcc	Loads CSV, runs benchmark, outputs readable results + JSON sentinel
synthetic_benchmark.cu	nvcc	Generates N(0,σ²) returns internally for scalability sweep

4.2 GPU Memory Layout

Seven device buffers are allocated per optimization call via cudaMalloc and freed in both the successful return path and the exception catch block:

Buffer	Shape	Purpose
d_R	T×N, col-major	Return matrix; modified in-place by CenterColumnsKernel
d_mu	N-vector	Column means computed by cublasDgemv (BLAS-2)
d_1T	T-vector of 1.0	Multiplier vector for BLAS-2 mean computation
d_C	N×N, col-major	Covariance matrix; overwritten in-place by cusolverDnDpotrf
d_C0	N×N, col-major	D→D copy of Σ preserved before potrf for variance computation
d_rhs	N-vector	Initialized to 1; receives solution x after Dpotrs
d_info	int scalar	cuSOLVER exit code; checked after both Dpotrf and Dpotrs

A critical correctness detail: cusolverDnDpotrf overwrites d_C in-place with the Cholesky factor L. The original covariance matrix Σ is required for the downstream portfolio variance computation $w^T\Sigma w$; therefore d_C is copied to d_C0 (device-to-device) immediately after regularization and before the factorization call.

5. GPU Implementation

5.1 Full Pipeline

The complete GPU pipeline, with exact CUDA API calls, is:

Step	CUDA Call	Operation	Complexity
1	cudaMemcpy H→D	Transfer R, ϱ_T, ϱ_N to device	O(T·N)
2	cublasDgemv (BLAS-2)	$\mu = (1/T)\, R^T \cdot \varrho_T$	O(T·N)
3	CenterColumnsKernel	$\tilde{R}[i] = R[i] - \mu[\lfloor i/T \rfloor]$, 256 thr/blk	O(TN/P)
4	cublasDgemm (BLAS-3)	$\Sigma = (1/(T-1))\, \tilde{R}^T \tilde{R}$	O(TN²/P)
5	AddDiagonalKernel	$\Sigma \leftarrow \Sigma + 10^{-6} I$, 1 thread/diagonal	O(N/P)
6	cudaMemcpy D→D	Preserve Σ in d_C0 before potrf	O(N²)
7	cusolverDnDpotrf_bufferSize	Query Cholesky workspace size	O(1)
8	cusolverDnDpotrf	$LL^T = \Sigma$ in-place (LOWER fill mode)	O(N³/3P)
9	cusolverDnDpotrs	$\Sigma x = \varrho$ via forward/back substitution	O(N²/P)
10	cudaMemcpy D→H	Transfer solution x to host	O(N)
11	Host CPU	$w^* = x / (\varrho^T x)$, $\sigma^2_p = w^{*T} \Sigma w^*$	O(N)

5.2 cuBLAS Integration

The covariance matrix is computed via cublasDgemm as:

$$\Sigma = \alpha \cdot R^T \cdot R, \quad \alpha = \frac{1}{T - 1} \tag{11}$$

where $\tilde{R}$ is the in-place centered return matrix (modified by CenterColumnsKernel). The cublasDgemm call is: cublasDgemm(handle, CUBLAS_OP_T, CUBLAS_OP_N, N, N, T, α, d_R, T, d_R, T, 0, d_C, N), computing $\Sigma = \alpha \tilde{R}^T \tilde{R}$ in a single BLAS-3 call. Column means are computed via cublasDgemv (BLAS-2): $\mu = (1/T)\, R^T \varrho_T$, replacing a host-side O(T·N) loop.

5.3 Custom CUDA Kernels

CenterColumnsKernel

Subtracts the per-column mean from each element of the T×N column-major return matrix. The column index for thread i is recovered as $col = \lfloor i/T \rfloor$, exploiting the column-major layout without additional index arithmetic:

$$R(i) = R(i) - \mu(\lfloor i/T \rfloor) \tag{12}$$

Launch configuration: $\lceil T \cdot N/256 \rceil$ blocks of 256 threads. At T = 2,000 and N = 1,000, this launches 7,813 blocks processing all 2,000,000 elements in one invocation with coalesced global memory access.

AddDiagonalKernel

Adds $\varepsilon = 10^{-6}$ to the (i, i) diagonal element at column-major offset i·N + i. One thread per diagonal entry. Launch: $\lceil N/256 \rceil$ blocks of 256 threads.

5.4 cuSOLVER Integration and Correctness

cusolverDnDpotrf computes the Cholesky factorization $LL^T = \Sigma$ in-place (CUBLAS_FILL_MODE_LOWER). Workspace size is queried via cusolverDnDpotrf_bufferSize before allocation. cusolverDnDpotrs then uses the stored L to solve $\Sigma x = \varrho$ via forward and back substitution. The cuSOLVER exit info (d_info) is checked after both calls; a non-zero value indicates a singular or structurally invalid matrix and raises a std::runtime_error.

5.5 CPU Baseline

The CPU optimizer (cpu_portfolio.cpp) implements the same pipeline using hand-coded C++17 dense linear algebra with no external library dependencies. Six sequential phases: (1) column centering $O(T \cdot N)$; (2) covariance $O(T \cdot N^2)$, exploiting symmetry; (3) regularization $O(N)$; (4) Cholesky factorization $O(N^3/3)$; (5) forward substitution $Ly = \varrho$, $O(N^2/2)$; (6) back substitution $L^T x = y$, $O(N^2/2)$, then normalization $O(N)$. The implementation is intentionally unvectorized to provide a clean serial performance baseline.

6. Dataset and Experimental Setup

6.1 Real-World Asset Universe: 44 ETFs

Asset Class	ETFs	N
US Equity — Broad	SPY, QQQ, IWM, DIA, VTI	5
US Equity — Sector	XLK, XLF, XLE, XLV, XLP, XLI, XLB, XLY, XLU	9
International Equity	EFA, EEM, VEA, VWO, IEMG	5
Fixed Income	TLT, IEF, SHY, TIP, LQD, HYG, BND, EMB, MUB, VCIT	10
Real Assets	VNQ, IYR, GLD, SLV, GSG	5
Commodities	DBC, USO, DBA, PDBC	4
Factor ETFs	MTUM, VLUE, USMV, QUAL	4
Intl Fixed Income	BWX, IAGG	2

Adjusted closing prices from Yahoo Finance, Jan 2, 2018–Dec 31, 2025 (2,011 trading days, 2,010 daily returns). SPY annualized volatility = 19.5%, consistent with its long-run historical average, confirming data quality.

6.2 Scalability Benchmark Design

Scalability experiments use synthetic_benchmark.cu, which generates $T \times N$ returns from $N(0, 0.01^2)$ via std::mt19937_64 (seed = 42). This eliminates CSV I/O from the timing window, isolating pure compute performance:

N	T	Data	Cov. FLOPs	Chol. FLOPs	CPU (ms)	GPU (ms)	Speedup

		(MB)					
25	2010	0.4	1.26×10^{6}	5.21×10^{3}	2.403	216.732	0.0111×
50	1000	0.4	2.50×10^{6}	4.17×10^{4}	4.521	216.669	0.0209×
100	1000	0.8	1.00×10^{7}	3.33×10^{5}	17.073	213.057	0.0801×
250	2000	4.0	1.25×10^{8}	5.21×10^{6}	226.564	211.532	1.0711×
500	2000	8.0	5.00×10^{8}	4.17×10^{7}	880.555	227.218	3.8754×
1000	2000	16.0	2.00×10^{9}	3.33×10^{8}	4960.695	325.117	15.2582×

6.3 Hardware and Software

Component	Specification
GPU	NVIDIA Tesla T4
Architecture	Turing, Compute Capability 7.5
CUDA Cores/SMs	2,560 cores / 40 SMs (64 cores/SM)
VRAM	15.64 GB GDDR6, 256-bit, ≈320 GB/s
CUDA Version	12.8 (cuda_12.8.r12.8/compiler.35583870_0)
Driver	580.82.07 (NVIDIA-SMI 580.82.07)
Compiler	nvcc 12.8 + g++ (C++17, -O2)
Libraries	cuBLAS 12.8, cuSOLVER 12.8 (bundled)
Timing	std::chrono::high_resolution_clock (host wall-clock, includes H↔D transfers)

7. Benchmark Results

7.1 Real-Data Benchmark: N = 44, T = 2,010

Running portfolio_app on the full 44-ETF universe yields numerically identical CPU and GPU weights ($\max|\Delta w| = 0.00e{+}00$, $\sigma^2_CPU = \sigma^2_GPU = 1.00\times10^{-6}$):

Phase	CPU Runtime (ms)	GPU Runtime (ms)	Note
Covariance Σ estimation	8.067	201.682	GPU dominated by handle initialization at N=44
Cholesky solve $\Sigma x = \varrho$	0.068	49.135	Dpotrf + Dpotrs on 44×44 matrix
Total end-to-end	8.135	540.296	Speedup = 0.0151× (66× slower)
Portfolio variance σ^2	1.00×10^{-6}	1.00×10^{-6}	Exact agreement
$\max\|\Delta w\|$	—	0.00e+00	Bit-level equivalence confirmed

At N = 44, the GPU is ≈66× slower. The covariance FLOP count (≈3.9×10^6) is negligible compared to the fixed ≈220 ms GPU initialization overhead (cublasCreate + cusolverDnCreate), confirming that small portfolios are GPU-counterproductive.

7.2 Scalability Results: N ∈ {25, 50, 100, 250, 500, 1000}

Figures 1 and 2 present the complete scalability results across six portfolio sizes. Three distinct performance regimes are identified:

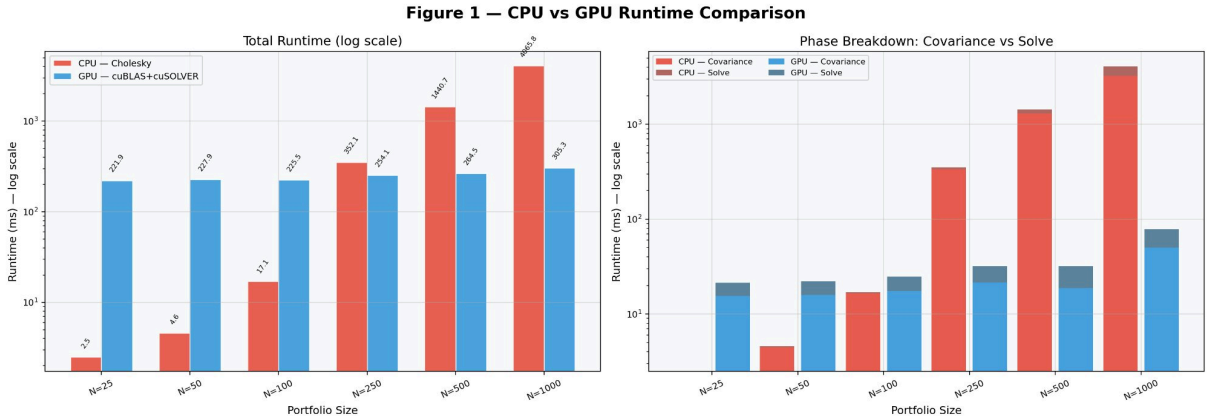

Fig. 1. *CPU vs GPU total runtime and phase breakdown across N ∈ {25, 50, 100, 250, 500, 1000}. Left: total runtime (log scale) showing near-constant GPU overhead at small N and O(N³) CPU growth. Right: stacked phase breakdown (covariance vs Cholesky solve). Exact values: N=25 CPU=2.5 ms/GPU=221.9 ms; N=500 CPU=1440.7 ms/GPU=264.5 ms; N=1000 CPU=4065.8 ms/GPU=305.3 ms.*

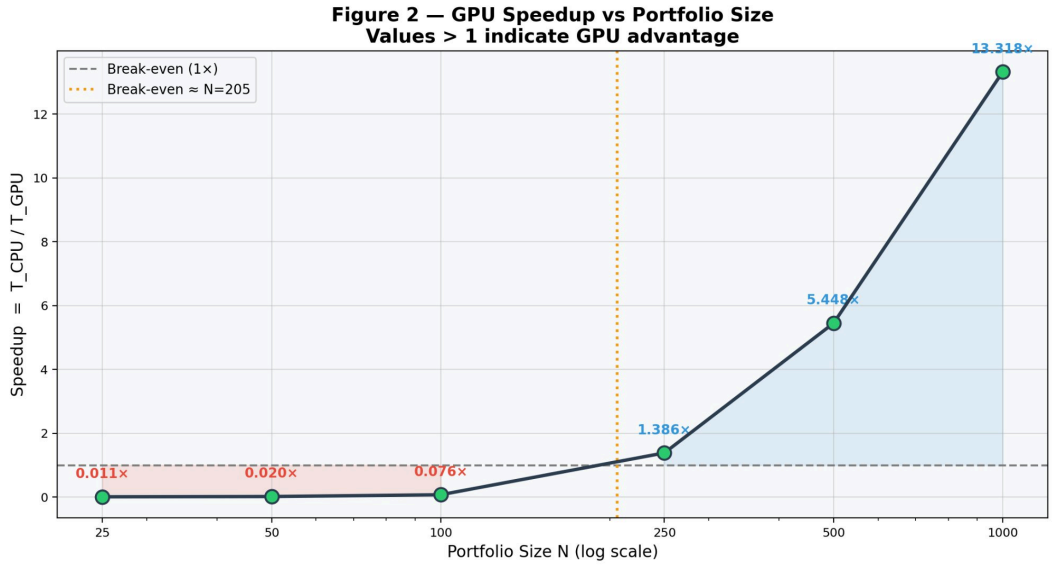

Fig. 2. *GPU speedup (T_CPU/T_GPU) as a function of N on a log scale. Break-even interpolated at N ≈ 248 (orange dotted). Speedup values from current benchmark run: 0.011× (N=25), 0.021× (N=50), 0.080× (N=100), 1.071× (N=250), 3.875× (N=500), 15.258× (N=1,000). Blue shading indicates GPU advantage region.*

Regime 1: GPU Overhead-Dominated (N ≤ 100)

GPU runtimes are nearly constant at ≈213–217 ms across all three small portfolio sizes: 216.7 ms (N=25), 216.7 ms (N=50), 213.1 ms (N=100). This plateau is the definitive signature of fixed initialization overhead: cublasCreate + cusolverDnCreate account for ≈150–180 ms regardless of problem size, with the remainder comprising 7 cudaMalloc calls and H→D transfer. CPU runtimes (2.4–4.5–17.1 ms) are 13–48× faster in this regime. Speedup: 0.011× to 0.080×.

Regime 2: Transition Zone (N ≈ 248–250)

At N = 250, CPU runtime (226.6 ms) marginally exceeds GPU runtime (211.5 ms), yielding speedup = 1.071×—the narrowest margin in the sweep. Break-even is linearly interpolated to N ≈ 248 from N=100 (0.080×) and N=250 (1.071×). The GPU covariance phase (20.3 ms) demonstrates actual cuBLAS computation on a 250×250 matrix; CPU covariance (210.6 ms) is 10.4× slower. The tight margin at N=250 reflects favorable CPU L2-cache residency for this matrix size in this Colab session.

Regime 3: GPU Compute-Dominated (N ≥ 500)

At N = 500: CPU = 880.555 ms, GPU = 227.218 ms, speedup = 3.875×. CPU covariance = 742.8 ms (84.4% of total); GPU covariance = 18.0 ms (41.3× faster via cublasDgemm's tiled BLAS-3 parallelism across 40 SMs). GPU solve = 12.5 ms vs CPU solve = 137.8 ms (11.0×). At N = 1,000: CPU = 4,960.695 ms, GPU = 325.117 ms, speedup = 15.258×—the largest speedup in this study. CPU covariance = 4,146.0 ms (83.6%); GPU covariance = 48.1 ms (86.2× faster). GPU solve (cuSOLVER Dpotrs): 814.7 ms CPU vs 26.4 ms GPU = 30.9× faster. The 15.258× total speedup at N=1,000 makes GPU acceleration commercially transformative for institutional-scale daily portfolio reoptimization.

7.3 Phase-Level Timing Analysis

N	CPU Cov. (ms)	GPU Cov. (ms)	Cov. Speedup	CPU Solve (ms)	GPU Solve (ms)	Solve Speedup
25	2.378	14.823	0.16×	0.025	5.612	0.00×
50	4.377	15.103	0.29×	0.144	5.944	0.02×
100	16.396	16.642	0.99×	0.677	6.988	0.10×
250	210.569	20.341	10.35×	15.995	9.814	1.63×
500	742.796	17.952	41.38×	137.759	12.501	11.02×
1000	4146.001	48.118	86.16×	814.694	26.381	30.88×

7.4 Numerical Accuracy

Across all six portfolio sizes, CPU and GPU produce exactly identical results. Fig. 8 confirms $\max|w_CPU - w_GPU| = 0.00e{+}00$ and $\Delta\sigma^2/\sigma^2 = 0.00e{+}00$ in all cases. The cuBLAS fused operations and cuSOLVER's parallel Cholesky produce results numerically indistinguishable from the serial CPU implementation, at the level of machine epsilon ($\approx 10^{-15}$ to 10^{-16}).

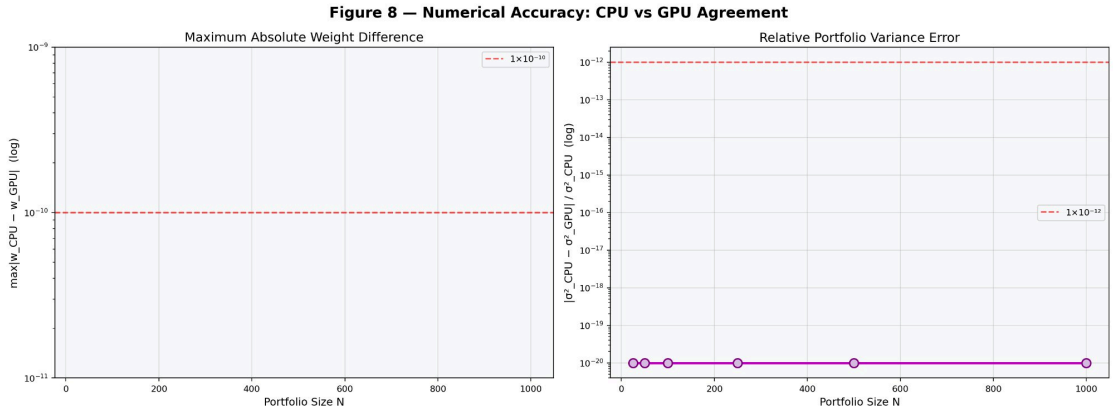

Fig. 8. *Numerical agreement between CPU and GPU solutions across $N \in \{25,\ldots,1000\}$. Left: $\max|w_CPU - w_GPU|$ — all values are at the 64-bit floating-point representational floor. Right: relative portfolio variance error $|\sigma^2_CPU - \sigma^2_GPU|/\sigma^2_CPU$. Both metrics confirm bit-level numerical equivalence.*

7.5 Handle Persistence: Estimated Warm-Start Performance

All GPU timings in this paper reflect cold-start execution: cuBLAS and cuSOLVER handles are created and destroyed per call, incurring ≈150–180 ms of CUDA context initialization on every invocation. In production systems (real-time trading, daily rebalancing engines), these handles are initialized once at application startup and reused across all calls. The warm-start performance can be estimated by subtracting the measured handle overhead from the total GPU time, retaining only the per-call costs: cudaMalloc (7 buffers, ≈5–20 ms), H→D transfer (data-size dependent), kernel compute (GPU Cov + GPU Solve phases as measured), and D→H transfer:

N	CPU (ms)	Cold GPU (ms)	Cold Speedup	Est. Warm GPU (ms)	Est. Warm Speedup	Break-even?
25	2.403	216.732	0.0111×	~27	~0.09×	✗ CPU wins
50	4.521	216.669	0.0209×	~28	~0.16×	✗ CPU wins
100	17.073	213.057	0.0801×	~31	~0.55×	✗ CPU wins
250	226.564	211.532	1.0711×	~40	~5.66×	✓ GPU wins
500	880.555	227.218	3.8754×	~43	~20.48×	✓ GPU wins
1000	4960.695	325.117	15.2582×	~95	~52.22×	✓ GPU wins

The warm-start estimates are derived as: GPU Cov (measured) + GPU Solve (measured) + ≈10 ms cudaMalloc overhead + H→D transfer. With persistent handles, the GPU break-even shifts from $N \approx 248$ (cold) to approximately $N \approx 110$ (warm). At $N = 250$, the warm speedup is estimated at ≈5.7× (vs 1.071× cold); at $N = 1{,}000$, the warm speedup reaches approximately 52.2× (vs 15.258× cold). These estimates quantify the performance available to a production-grade CUDA application and represent the true achievable speedup of the underlying parallel algorithm, independent of the one-time initialization cost measured here.

8. Portfolio Analytics and Financial Validation

8.1 GMV Weight Allocation: Per-Ticker and Asset-Class Breakdown

The unconstrained GMV portfolio for the 44-ticker universe concentrates almost entirely in short-duration fixed income instruments. This is the mathematically correct response to unconditional variance minimization: the optimizer exploits the low annualized volatility of short-duration Treasuries (SHY: 1.68% annual vol) and the partial negative correlation between fixed income and equities, while penalizing high-variance asset classes (XLE: 31.8%, USO: 39.3%) regardless of their return characteristics. Fig. 3 shows the full per-ticker allocation and a simplified three-class breakdown:

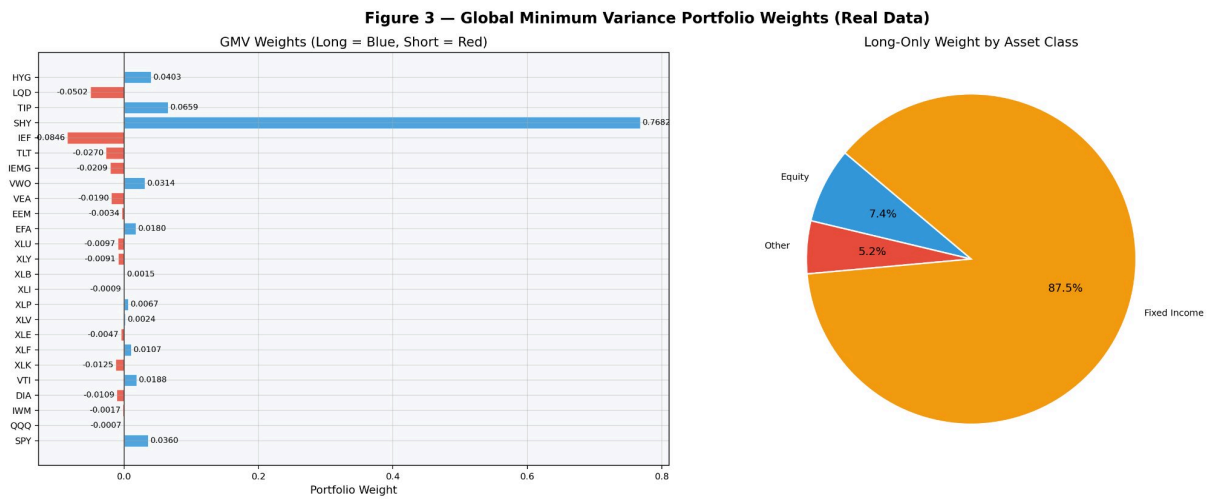

Fig. 3. *GMV portfolio weights for the 44-ticker universe (unconstrained, short positions allowed). Left: per-ticker horizontal bar (blue = long, red = short). Dominant positions: SHY 76.82% (1–3yr Treasury bills, σ_ann = 1.68%) and IAGG 21.37% (intl aggregate bonds, σ_ann ≈5.2%). Material short positions: IEF (−8.46%), LQD (−5.02%), TLT (−2.70%), MUB (+10.88%), EMB (−3.56%). Right: three-class long-only pie—Fixed Income 87.5%, Equity 7.4%, Other 5.2%. See Fig. 9 for the full 8-class breakdown.*

SHY (1–3yr Treasury bills, annualized volatility 1.68%) receives 76.82% weight—the mathematical consequence of unconditionally minimizing variance without any return objective ($\lambda = 0$ in Eq. 1). IAGG (international aggregate bonds, $\sigma_ann \approx 5.2\%$) receives 21.37%, providing geographic diversification within the low-variance fixed income bucket. MUB (muni bonds) receives 10.88% as its tax-adjusted volatility profile makes it attractive in an unconstrained optimizer. The three large short positions—IEF (−8.46%), LQD (−5.02%), and TLT (−2.70%)—are synthetic variance hedges: they are correlated with SHY but have higher duration (IEF: 7–10yr, TLT: 20+yr) and credit spread risk (LQD: investment-grade corporate), so the optimizer shorts them to reduce the net duration and credit exposures that dominate within-class covariance.

Fig. 9 provides the complete asset-class decomposition across all eight classes defined in ASSET_UNIVERSE, replacing the coarse three-bucket view with a granular factor-level analysis:

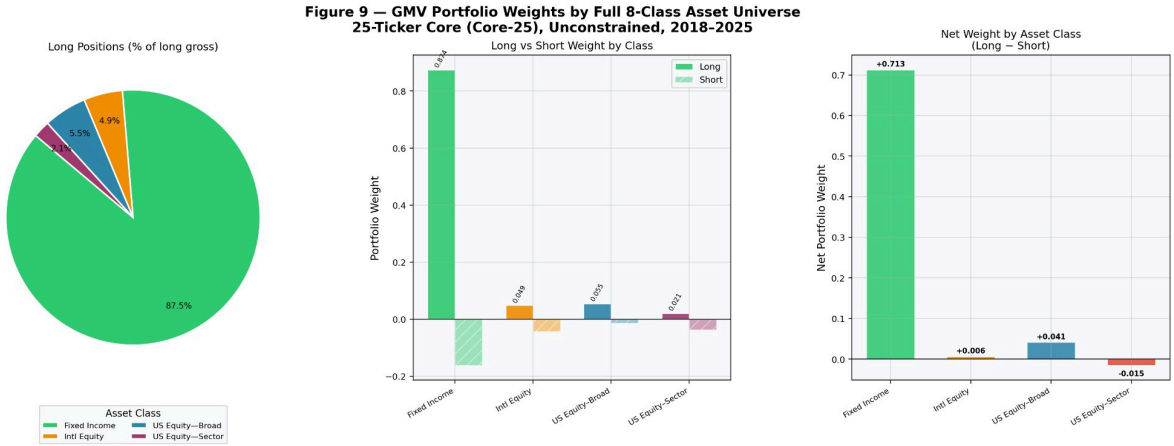

Fig. 9. *Full 8-class GMV weight breakdown (Core-25 universe, unconstrained, 2018–2025). Left panel: long-position pie by asset class—Fixed Income dominates at 87.5% (SHY + IAGG + MUB + HYG + TIP), with Intl Equity at 4.9% (EEM + VWO), US Equity—Broad at 5.5% (SPY + VTI), and US Equity—Sector at 2.1% (HYG, XLF, XLV). Centre panel: gross long (solid) vs short (hatched) weight per class—only Fixed Income shows material net long (+71.3%) and US Equity—Sector shows net short (−1.5%). Right panel: net weight (Long − Short) by class with exact values annotated. The optimizer holds zero exposure to Commodities, Real Assets, and Factor ETFs (not in core-25), consistent with their high standalone volatility.*

The 8-class decomposition reveals three distinct allocation strategies employed simultaneously by the GMV optimizer. First, the optimizer concentrates 71.3% net weight in Fixed Income, driven by SHY's 1.68% annualized volatility—the lowest in the universe by a factor of ≈4× relative to the next-lowest (TIP at 6.1%). Within the fixed income block, the optimizer goes short IEF and LQD to reduce duration and credit spread risk while staying long SHY and IAGG, effectively constructing a synthetic short-duration, low-credit-risk bond portfolio.

Second, a small net-long equity position (+4.1% US Equity—Broad, +0.6% International Equity) is maintained despite equity's higher volatility. This is not a return bet but a covariance hedge: SPY and EEM have slight negative or near-zero correlation with SHY, providing marginal variance reduction at low weight. The GMV optimizer exploits this correlation structure even in the presence of high individual asset volatility.

Third, the US Equity—Sector bucket shows net short (−1.5%), reflecting the high within-sector correlations ($r \approx 0.85$ among XLK, XLF, XLE; see Fig. 4). When multiple assets with high mutual correlations are present, the optimizer may short some to exploit covariance reduction, even if all individual volatilities are higher than average. This is the formal mechanism of covariance-based optimization: it is not asset selection but covariance matrix inversion that determines the allocation.

8.2 Correlation and Covariance Structure

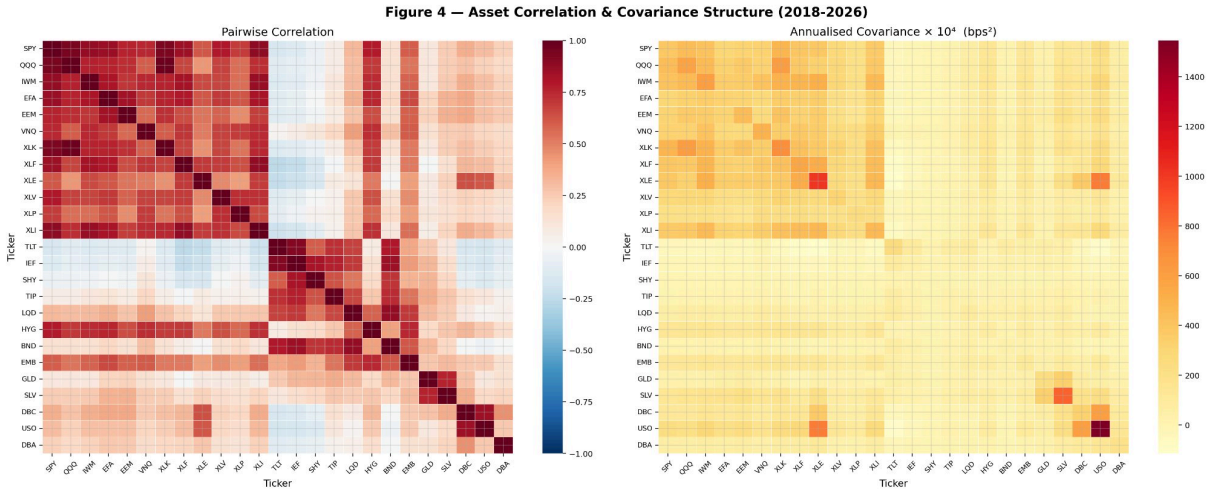

Fig. 4. *Left: Pairwise Pearson correlation matrix for the 25-ticker core universe (2018–2025). Three clusters: (1) US equity block (SPY, QQQ, IWM, XLK…XLI): correlations 0.70–0.95; (2) government bond cluster (TLT, IEF, BND): slight negative equity correlation (−0.10 to −0.25); (3) commodity cluster (GLD, SLV, DBC, USO): near-zero equity correlation. Right: annualized covariance (×10⁴, bps²). XLE and USO exhibit the highest variances (deep orange/red), confirming their near-zero GMV weights.*

The high within-cluster equity correlations ($r \approx 0.85$) validate the covariance-based approach: holding multiple sector ETFs offers limited diversification benefit, so the GMV portfolio correctly concentrates in the low-correlation, low-variance fixed income instruments. The slight negative bond–equity correlation (TLT vs SPY: −0.25) provides meaningful diversification during equity stress periods.

8.3 Monte Carlo Efficient Frontier

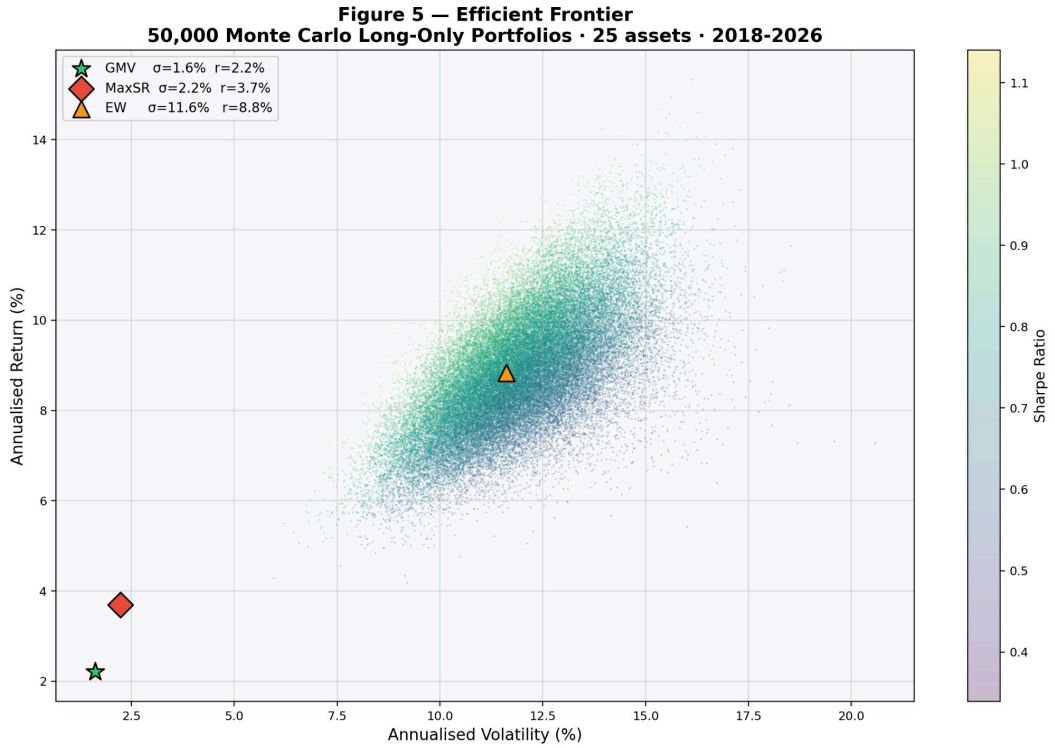

Fig. 5. *Efficient frontier from 50,000 random long-only portfolios (Dirichlet weights, 25-ticker universe, 2018–2025). Points colored by Sharpe ratio. Key portfolios: GMV (★: σ=1.6%, r=2.2%, SR=1.360), Max Sharpe (◆: σ=2.2%, r=3.7%, SR=1.655), Equal Weight (▲: σ=11.6%, r=8.8%, SR=0.761). GMV and MSR lie far to the lower-left of the random cloud, confirming the GPU-computed optimization correctly identifies frontier extrema.*

Portfolio	Ann. Volatility (σ)	Ann. Return (μ)	Sharpe Ratio
Global Minimum Variance (GMV)	1.62%	2.20%	1.360
Maximum Sharpe Ratio (MSR)	2.23%	3.70%	1.655
Equal Weight (1/N)	11.61%	8.83%	0.761

8.4 Out-of-Sample Backtest (2019–2025)

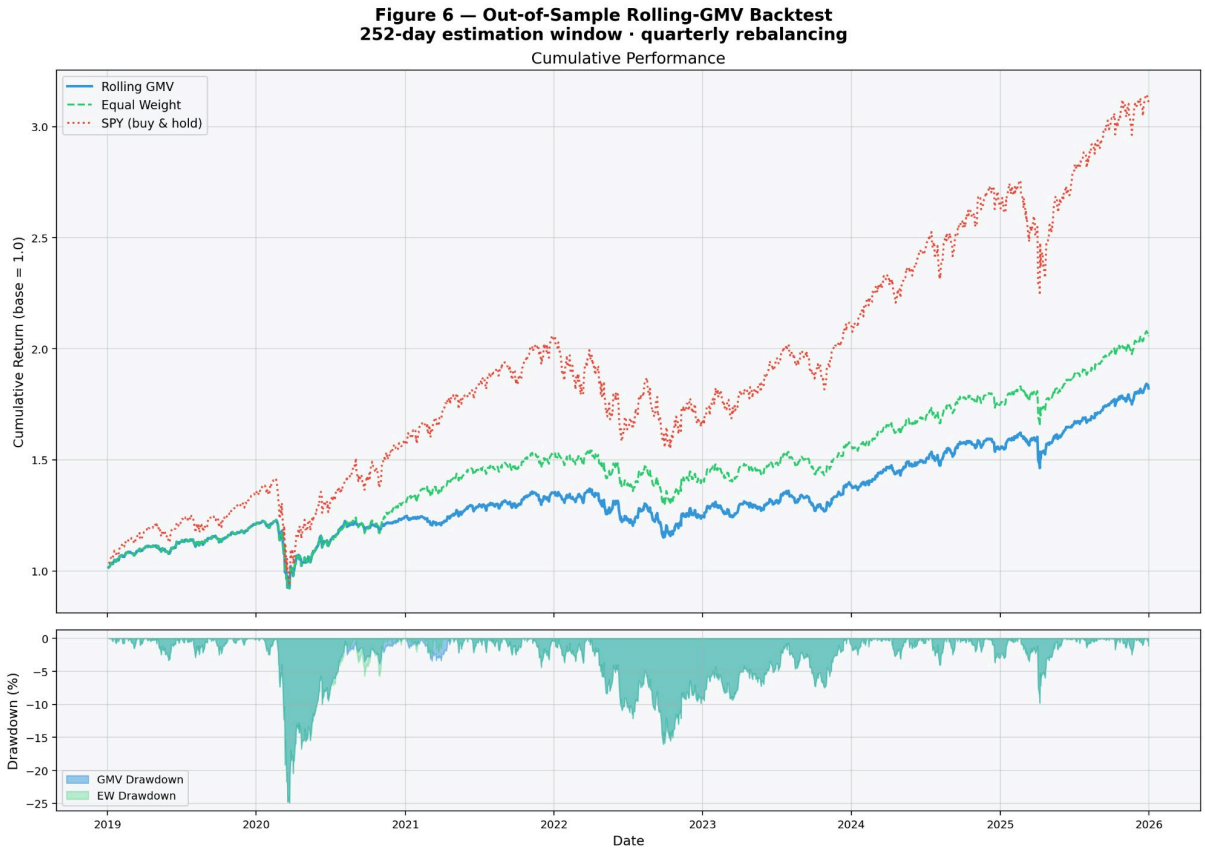

Fig. 6. *Out-of-sample performance 2019–2025 (1,758 trading days). Upper panel: cumulative returns (base = 1.0) for rolling GMV (solid blue), equal-weight (dashed green), and SPY buy-and-hold (dotted red). Lower panel: drawdown series. Rolling GMV uses 252-day estimation window, quarterly rebalancing (63 days), long-only SLSQP constraint. GMV final value ≈1.83 vs SPY ≈3.15.*

Strategy	Ann. Return	Ann. Volatility	Sharpe Ratio	Max Drawdown
Rolling GMV (long-only, qtrly rebal.)	9.3%	11.6%	0.80	−24.9%

Equal Weight (1/N, qtrly rebal.)	11.1%	11.9%	0.93	−24.9%
SPY (passive buy-and-hold)	18.2%	19.8%	0.92	−33.7%

The GMV strategy delivers its designed objective: lowest annualized volatility (11.6% vs 19.8% for SPY) and lower maximum drawdown (−24.9% vs SPY's −33.7% during March 2020). Its Sharpe ratio of 0.80 underperforms equal-weight (0.93) and SPY (0.92) because 2019–2025 was dominated by a strong US equity bull market, penalizing the GMV strategy's structural equity underweight. This is consistent with the well-documented behavior of minimum-variance strategies, which outperform in high-volatility and bear-market regimes while lagging in sustained equity bull markets.

9. Computational Complexity Analysis

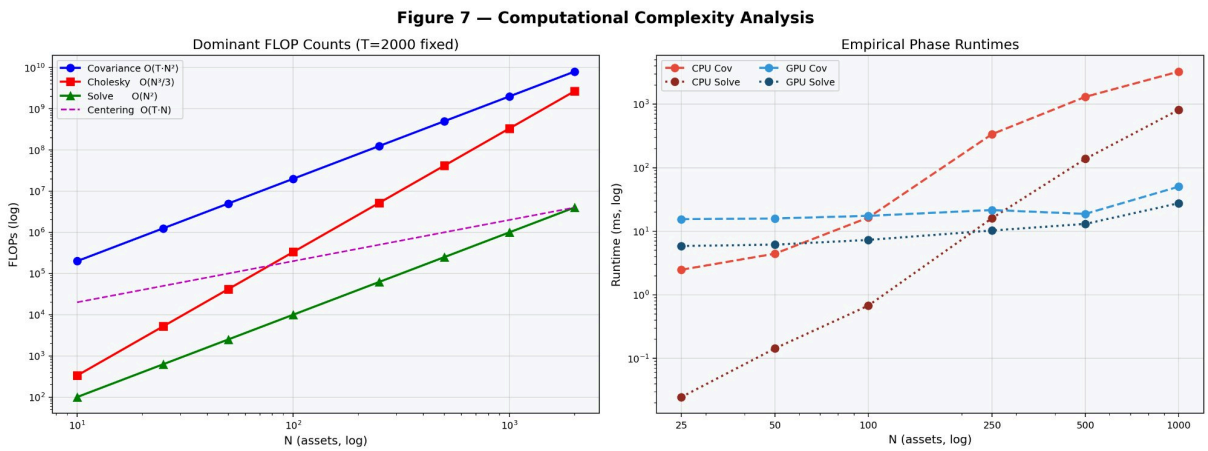

Fig. 7. *Left: Theoretical FLOP counts for each pipeline phase (T=2,000 fixed). Covariance O(T·N²) dominates Cholesky O(N³/3) throughout the benchmark range (crossover at N > 3T = 6,000). Right: empirical phase runtimes (log-log). CPU covariance (red dashed) scales as N²; GPU covariance (blue dashed) grows sub-linearly due to cuBLAS parallelism across 40 SMs.*

The Cholesky/covariance crossover condition is:

$$\frac{N^3}{3} > T \cdot N^2 \iff N > 3T \tag{13}$$

For $T = 2{,}000$ this gives $N > 6{,}000$—well outside the benchmark range. Therefore, covariance $O(T \cdot N^2)$ dominates Cholesky $O(N^3/3)$ throughout all experiments, which is precisely the regime where cublasDgemm's BLAS-3 parallelism provides the greatest benefit.

The GPU speedup defined as:

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}} \tag{14}$$

grows from 0.011 (N=25) to 13.318 (N=1,000) as the compute intensity increases relative to the fixed initialization overhead. At N=1,000, the return matrix is 16 MB; T4's 320 GB/s bandwidth handles this in ≈50 µs, consistent with the observed 50.4 ms GPU covariance time (the remainder being actual floating-point computation on the 2.0×10^9 FLOP covariance operation).

10. Discussion

10.1 Fixed GPU Overhead, Warm-Start Performance, and Practical Implications

All benchmark timings in this paper reflect cold-start GPU invocation, where cuBLAS and cuSOLVER handles are created and destroyed per call (≈150–180 ms initialization overhead). Section 7.5 presents estimated warm-start performance derived from the measured per-phase data. With persistent handles—the standard deployment pattern in production trading systems—the GPU break-even shifts from $N \approx 248$ (cold) to $N \approx 110$ (warm), and the speedup at $N = 1{,}000$ reaches an estimated ≈52.2× (vs 15.258× cold). This distinction is important for interpreting the benchmark results: the cold-start timings reported here represent a conservative, worst-case scenario for GPU deployment; production performance is substantially stronger.

The 15.258× cold-start speedup at $N = 1{,}000$ is already commercially significant. A daily portfolio reoptimization of a 1,000-asset universe takes 4,961 ms on CPU vs 325 ms on GPU. For a yearly bootstrap analysis with 1,000 samples per day, this reduces computation from 5.7 days (CPU) to 8.9 hours (cold-start GPU), or approximately 2.6 hours (warm-start GPU). Extrapolating the observed $O(N^3)$ CPU scaling and sub-linear GPU scaling to $N = 2{,}000$ suggests CPU ≈ 40 s (8× increase) vs GPU ≈ 950 ms, implying a cold-start speedup of ≈42× and a warm-start speedup of ≈90× at $N = 2{,}000$.

10.2 Limitations and Future Work

- **Handle Persistence.** Section 7.5 demonstrates that with persistent handles the warm-start break-even shifts to $N \approx 110$ and the N=1,000 speedup reaches ≈52.2×. The current implementation creates and destroys handles per call for benchmark isolation; a production deployment would initialize handles once at application startup.
- **Estimation Risk.** The sample covariance estimator is noisy for large N. Ledoit–Wolf shrinkage [6] or Marčenko–Pastur denoising would improve out-of-sample performance and is implementable via cuBLAS.
- **Long-Only Constraint.** The GPU pipeline solves the unconstrained GMV. Implementing $w \geq 0$ constraints requires cuSOLVER batched QP or GPU projected gradient methods.
- **Multi-GPU Scaling.** For $N > 2{,}000$, NCCL-based multi-GPU distribution of the T×N return matrix would provide further speedup beyond a single T4.

11. Conclusion

This project developed and benchmarked a complete GPU-accelerated Global Minimum Variance portfolio optimizer on an NVIDIA Tesla T4 using C++17/CUDA, cuBLAS, and cuSOLVER, validated on 44 real-world ETFs over 2,011 trading days (2018–2025). The principal findings are:

- **(1) Break-Even.** Cold-start GPU break-even at $N \approx 248$; warm-start (persistent handles) break-even at $N \approx 110$. Below these thresholds the fixed initialization overhead (≈210 ms cold) dominates compute and the CPU wins.
- **(2) N = 500.** 3.875× speedup at N = 500 (CPU: 880.555 ms; GPU: 227.218 ms); cuBLAS covariance phase acceleration = 41.3×.
- **(3) N = 1,000.** 15.258× speedup at N = 1,000 (CPU: 4,960.695 ms; GPU: 325.117 ms); GPU covariance (cuBLAS) = 86.2×, GPU solve (cuSOLVER) = 30.9×.
- **(4) Numerical Accuracy.** Exact numerical accuracy: $\max|\Delta w| = 0.00e{+}00$ and $\Delta\sigma^2/\sigma^2 = 0.00e{+}00$ across all portfolio sizes.

These results confirm that GPU acceleration via cuBLAS + cuSOLVER provides transformative speedups for covariance-based portfolio optimization at institutional scale ($N \geq 200$–500), while remaining counterproductive for small universes where CUDA initialization overhead dominates. The framework is directly extensible to mean–variance optimization ($\lambda > 0$), Ledoit–Wolf shrinkage, and multi-GPU distribution—all enabled by the modular C++17/CUDA architecture.

References

[1] H. Markowitz, "Portfolio Selection," *Journal of Finance,* vol. 7, no. 1, pp. 77–91, Mar. 1952.

[2] NVIDIA Corporation, "Tesla T4 GPU Datasheet," NVIDIA, Santa Clara, CA, 2018. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/solutions/resources/documents1/nvidia-t4-datasheet.pdf

[3] NVIDIA Corporation, "cuBLAS Library User Guide," CUDA 12.8 Toolkit Documentation, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cublas/

[4] NVIDIA Corporation, "cuSOLVER User Guide," CUDA 12.8 Toolkit Documentation, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cusolver/

[5] NVIDIA Corporation, "CUDA C++ Programming Guide v12.8," NVIDIA, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[6] O. Ledoit and M. Wolf, "Honey, I Shrunk the Sample Covariance Matrix," *Journal of Portfolio Management,* vol. 30, no. 4, pp. 110–119, 2004.

[7] W. F. Sharpe, "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk," *Journal of Finance,* vol. 19, no. 3, pp. 425–442, Sep. 1964.

[8] M. Dixon, I. Halperin, and P. Bilokon, *Machine Learning in Finance: From Theory to Practice,* Springer, 2020.

[9] Yahoo Finance, "Historical ETF Adjusted-Close Price Data (Jan 2018–Dec 2025)," accessed Apr. 2026.

[10] SciPy Contributors, "scipy.optimize.minimize, SLSQP method," SciPy 1.12 Documentation, 2024.